

HDOC Day_26 Programming in C

Problem

Write a C program that displays the following menu and uses a switch statement to print the “Right-aligned

patterns” selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate

parameterless function with no return value.

Note: For each activity, prepare the algorithm and test-case table. Then write, compile and execute the C program,

and test it against the prepared test cases.

1. Right-aligned increasing triangle

Input the number of rows, n, and print a right-aligned increasing triangle using asterisks.

For n = 4:

```
      *
     **
    ***
   ****
```

2. Right-aligned increasing number triangle

Input the number of rows, n, and print a right-aligned triangle of natural numbers. In each row, print the natural

numbers from 1 up to the corresponding row number.

For n = 4:

```
      1
     1 2
    1 2 3
   1 2 3 4
```

3. Right-aligned decreasing triangle

Input the number of rows, n, and print a right-aligned decreasing triangle using asterisks.

For n = 4:

* * * *

* * *

* *

*

Algorithm

Here is the step-by-step algorithm corresponding directly to your provided C program:

Main Algorithm

1. Start.
2. Display the menu options for choices 1, 2, and 3.
3. Read the user's choice into variable ch.
4. Use a switch statement on ch:
 - **Case 1:** Call function rait()
 - **Case 2:** Call function raint()
 - **Case 3:** Call function radt()
 - **Default:** Print "invalid input"
5. End.

Sub-Algorithm 1: rait() (Right-Aligned Increasing Asterisk Triangle)

1. Prompt and read the number of rows (n).
2. Loop i from 1 to n (controls rows):
 - Set z = n - i (number of leading spaces).
 - Loop j from 1 to n (controls column positions):
 - If j <= z, print two spaces ().
 - Otherwise, print an asterisk and space (*).
 - Print a newline character (\n).

Sub-Algorithm 2: raint() (Right-Aligned Increasing Number Triangle)

1. Prompt and read the number of rows (n).
2. Loop i from 1 to n (controls rows):
 - Initialize x = 1 (starting natural number for the row).
 - Set z = n - i (number of leading spaces).
 - Loop j from 1 to n (controls column positions):
 - If $j \leq z$, print two spaces ().
 - Otherwise, print the current value of x followed by a space, then increment x (x++).
 - Print a newline character (\n).

Sub-Algorithm 3: radt() (Right-Aligned Decreasing Asterisk Triangle)

1. Prompt and read the number of rows (n).
2. Loop i from 1 to n (controls rows):
 - Loop j from 1 to n (controls column positions):
 - If $j < i$, print two spaces ().
 - Otherwise, print an asterisk and space (*).
 - Print a newline character (\n).

C program code

```
#include <stdio.h>

void rait()
{
    int n;

    printf("enter no of rows\n");

    scanf("%d",&n);

    for(int i=1;i<=n;i++)
    {
```

```

        int z=n-i;
        for(int j=1;j<=n;j++)
        {
            if(j<=z)
                printf(" ");
            else
                printf("* ");
        }
        printf("\n");
    }
}

void raint()
{
    int n;
    printf("enter no of rows\n");
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        int x=1;
        int z=n-i;
        for(int j=1;j<=n;j++)
        {
            if(j<=z)
                printf(" ");
            else
            {
                printf("%d ",x);
                x++;
            }
        }
    }
}

```

```

    }
    printf("\n");
}
}
void radt()
{
    int n;
    printf("enter no of rows\n");
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n;j++)
        {
            if(j<i)
                printf(" ");
            else
                printf("* ");
        }
        printf("\n");
    }
}
int main()
{
    int ch;
    printf("choice 1 is to print a right-aligned increasing triangle using asterisks..\n");
    printf("choice 2 is to print a right-aligned triangle of natural numbers.\n");
    printf("choice 3 is to print a right-aligned decreasing triangle using asterisks\n");
    printf("enter your choice\n");
    scanf("%d",&ch);

```

```
switch (ch)
{
    case 1:
    {
        rait();
        break;
    }
    case 2:
    {
        raint();
        break;
    }
    case 3:
    {
        radt();
        break;
    }
    default:
    {
        printf("invalid input\n");
        break;
    }
}
return 0;
}
```